

Ella Potter & James Whipple

Professor Beichel

ECE:3360 Embedded Systems

Final Project Report

1 Introduction

1.a Motivation & Background

Our motivation for this project was to make a device that emulated the function of a simple compass. To achieve this, we used a MMC3416PJ, a three-axis AMR magnetometer that can detect magnetic fields. We wanted to make a handheld device that allows people to interface with the sensor and get magnetic field readings.

1.b Goals & Specifications

Our goal for this lab was to create a digital compass that could be used to determine both cardinal and rotational direction. We wanted our compass to be able to turn on and off and then when on display the cardinal direction and degrees of the compass to the user. The main idea of having the compass turn on and off is so the device does not use unnecessary power or resources if the user is not even interacting with it. The following list shows what components we needed to be able to implement the compass fully:

- Push-Button Switch: a push button switch will be used to easily turn the compass on and off. The compass should initially be off and turn on when the user presses the button, it should also turn off at anytime during execution if the user re-presses the button.
- Liquid Crystal Display: a LCD will be used to display the direction and degrees to the user when the compass is on. The LCD should be constantly refreshing its data if the direction or degrees from the compass changes.
- LED: four LED lights will be used to specifically point to the direction the compass is facing. (i.e. if the compass is facing north the very top light should be lit up)
- 3 axis Sensor: a MMC3416PJ will be used to detect the strength of magnetic fields on 3 axis's that then allows us to calculate a heading.
- MOSFET: two n-channel MOSFETS were used to act as digital level shifters to allow the I2C and the microcontroller to communicate properly

2 Implementation

2.a Overview

The digital compass contains five different hardware components, a push-button switch, a LCD, four LED's, a 3-axis magnetic sensor, and two MOSFETs. The 3-axis magnetic sensor and the two MOSFETs are primarily used in calculating the direction for the compass and sending the data back to the microcontroller. The push-button switch, LCD, and LEDS are all used in the display component of the compass and take the data from the 3 axis sensor and show it to the user. The software for the compass contains a C program containing all the main functions for controlling the compass, as well as two imported libraries to help with controlling the LCD and the 3 axis sensor.

2.b Hardware Description

The hardware for this project contained two different parts as mentioned before, so we decided to split the components onto two different breadboards (This also made it easier to move the compass component around). Therefore, we will split the following section into two parts, the display circuit and the compass circuit.

2.b.i Display Circuit

We began by creating the breadboard for all the display components as they were all components we had previously worked with. This board contained three components, the LEDs, the push-button switch, and the LCD:

1. **LED:** the LEDs were pretty straightforward to set up as we had worked with them before in Lab 1, the blinky lab. Since we wanted to use them to be able to physically show the cardinal direction we used 4 different ones, all of different colors. The red was set up to represent north, yellow east, green south, and blue represented west.

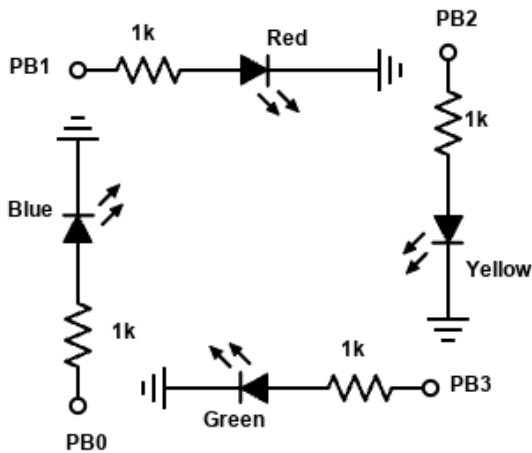


Figure 1: LED Circuit

Each LED was set up so that the cathode was connected to ground and the anode connected in series to a 1k resistor and PORTB. This allowed for any of the LEDs to be turned on by simply setting PORTB to 1 at whatever input the certain LED was hooked up to.

2. **Push-Button Switch:** the PBS was another component that was easily setup as it was used in multiple labs throughout the semester. This circuit just needed to be able to receive a pulse signal whenever the button was pushed and send that to the microcontroller to either turn the compass on or off.

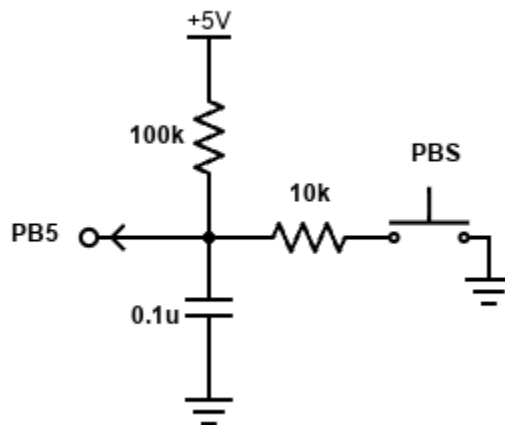


Figure 2: PBS Circuit

The circuit for the push-button was set up to correctly debounce the input signal sent to the microcontroller. When the button is not being pressed it will constantly send a logic

high signal, whereas when it is pressed it will send a logic low signal as the switch connects to ground. The 100k resistor is connected in parallel to the switch and is used as a pull-up resistor to produce the logic high output. The 0.1uF capacitor is also connected in parallel to the switch and helps to debounce the signal. Finally, the 10k resistor is connected in series with the switch and helps to limit the current flow from the switch to the microcontroller.

3. **LCD:** the final, and the most important, component in the display circuit was the LCD. We had used this component before in Lab 4, so we were able to use somewhat of the same circuit from that lab. Since we were now working in C instead of assembly language and would be using a library for the LCD, we added in the connection of the R/W pin.

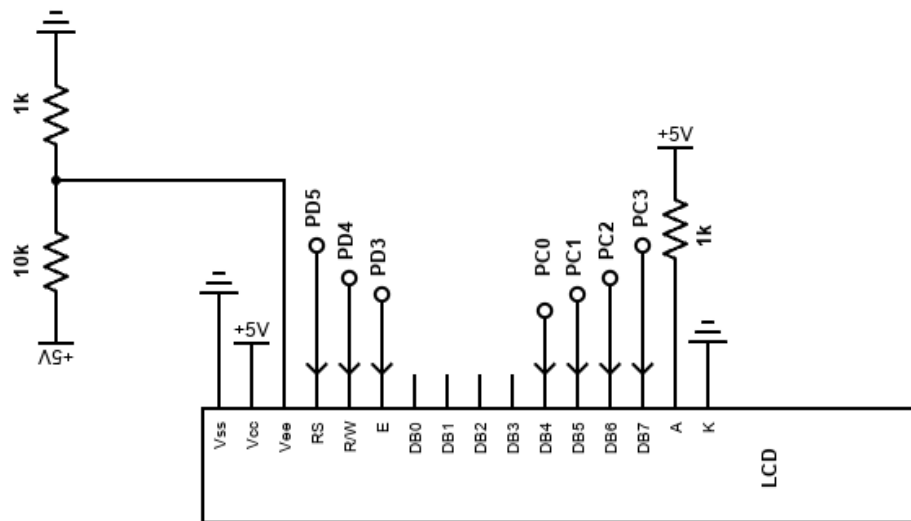


Figure 3: LCD Circuit

The pin connections for the LCD are explained below.

- Vss and Vcc go to ground and 5V to supply the power to the LCD.
- Vee is connected to a contrast voltage input by connecting both to a 10k pull up resistor and a 1k pull down resistor in parallel
 - This creates a voltage divider: $V = 5 * (1/(10+1)) = 0.45 \text{ V}$
- RS, R/W, and E are all connected to PORTD on the microcontroller and will be sent outputs from the uC.
 - RS will control whether the LCD is reading the input as data or as a command
 - R/W will control whether the LCD is going to read or write data
 - E is used to enable reading and writing to the LCD.

- DB4-DB7 are all the 4 data pins used in 4-bit mode and are all connected to PORTC on the uC and will be sent data from there
- A and K provide the backlight for the LCD and resemble an anode (A) and cathode (K) therefore A is connected to 5V and K is connected to ground

2.b.ii Compass Circuit: Working with the MMC3416 was the most difficult part of this project. The chip is a 3.3V device that is not tolerant to 5V, so we had to use a level shifter. We ended up making our own using n-channel MOSFET s and pullup resistors as described in the Mixed V lecture.

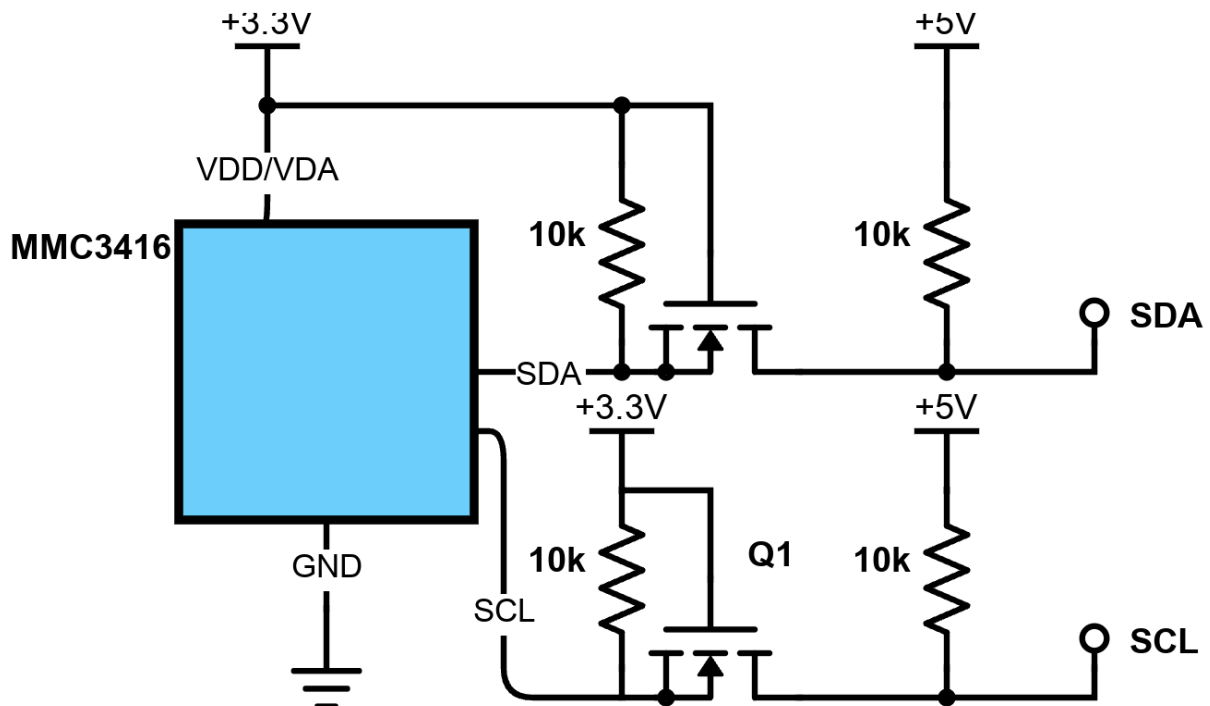


Figure 4: Compass Circuit

VDD is the voltage supply for the chip while VDA is the voltage reference for I2C, both are set at 3.3V. The 3.3V side of the I2C lines are connected to the source pin of the MOSFETs with a 10k Ohm pullup resistor connecting them to 3.3V. The gate pin is set to 3.3V and the 5V side of the I2C lines are connected to the drain end with a similar pullup resistor. This setup allows both sides to be pulled up independently, but will both be pulled low together, allowing to two-way I2C communication. The MOSFETs used were 2N7000's.

2.c Software Description

The software for the compass was written in C language, and consisted of a main program and two imported libraries. The software was split into two parts, and we first started by making sure the display circuit worked and implemented skeleton code for that before moving onto the compass code. In the end both the codes would work together to correctly interact with

the i2c device for the compass, receive the compasses data, and send that data to the LCD. The following will follow the same format as the Hardware Description section, breaking the code descriptions into two sections.

2.c.i Display Code

The software for the display component of the compass needed to implement the push-button the LEDs, and the LCD. Since we had not used any of these components before with C we had to try to convert our knowledge of how to work them in assembly to C. We were able to easily implement the button and LEDs, but the LCD looked like it may take a lot of time to write all the code, so we searched for a library and found one by Peter Fleury. The following will explain first the LCD library we used and then our main.c code for the button, LED, and LCDS.

2.c.i.i. LCD Library – Peter Fleury

The library we used from Fleury contained both a header and a source file for the LCD, lcd.h and lcd.c. The lcd.h file contains the skeleton code for the functions implemented in lcd.c as well as all the definitions of different components used in the LCD. In this file we only needed to change two things, we had to redefine all the LCD pins to the correct pins we were using in our circuit and also redefine the LCD initialization times for the certain LCD we were using. The following table will describe the different functions we used from lcd.c:

Function	Description
lcd_init(command)	This function takes a predefined command, representing what to do after initialization (display off/on, cursor off/on), and will correctly initialize the LCD and then do what the command says. To do this, it will first initialize all defined LCD ports as outputs. Next, it will run through the extensive process of initialization that is given in the LCD datasheets. Once initialized it will turn the display and cursor on or off depending on what command was given.
lcd_home(void)	This function will set the cursor on the LCD to the home position of (0,0). To do this it simply just sends the bits representing the home command to the LCD.
lcd_clrscr(void)	This function will clear the screen of the lcd and also set the cursor to home position. This is similar to the last function as it just sends the bits representing the clear command to the LCD.
lcd_gotoxy(x,y)	This function will set the cursor to a specific x y position on the LCD, which is taken as input. This will also send a command to the LCD specifying the start line using the given x and y values.
lcd_puts(string)	This function takes a string of characters and displays them on the LCD screen. This goes through the string character by character and places the characters at preceding positions on the LCD screen.

2.c.i.ii. Main Display Code

Once the LCD library was added into our project dependencies, we were ready to begin writing our main program to control all of the display functions of the compass. The following table describes the different functions in main.c and what they do:

Function	Description	Use
main	This is the main function that will be ran at the beginning of execution. It begins by setting the inputs and outputs on PORTD, PORTB, and PORTC. It then calls the turnOnLCD to initialize and turn on the LCD. After this, it contains a while(1) loop to continuously loop while the program is executing. This loop will continuously get PINB and check if bit 6 is low, meaning the PBS was pressed. If the PBS was pressed and the power of the compass is not already on it will turn the power on and wait for the button to be unpressed. Otherwise, if the PBS was pressed and power is already on then the compass will be turned off. This loop also, if the power is on, will continuously get the degrees and direction from the compass and if either one changes then it will update the LCD with the new data.	This function is ran once at beginning of execution, and then the while loop is continuously executed throughout the whole execution.
turnOnLCD	This function begins by initializing the LCD and turning the display and cursor on using the lcd_init function given by the LCD library. After initialization, it will clear the LCD screen and put the cursor in the home position. It will then display the message "PRESS BUTTON TO BEGIN" on the LCD screen.	This function is called at the beginning of execution, to tell the user how to begin the compass. It is also used anytime the compass is turned off to redisplay this string.
changeDegree	This function is used to change the degree value being displayed on the LCD. It takes an integer value representing the new degree value to be put on the LCD. It will transform this int to a string and then put "DEGREES: x" on the LCD.	This will be called anytime the system gets a new degree output from the compass. This ensures the LCD is not constantly updating unless it needs to be.
	This function is similar to changeDegree as it used to change the cardinal direction being displayed anytime it is updated. It takes in two integers, the current direction being displayed and the next	This is called anytime the degree or direction output

changeDirection	direction being displayed. It will first determine what the current direction is and turn on the correlated LED light (i.e. redLight(0)). After turning that light off it will determine what light to turn on for the next direction and turn that on (i.e. redLight(1)).	from the compass changes.
redLight, yellowLight greenLight blueLight	These four functions control the four LED lights on the display board. Each function takes in an integer value representing whether the light should be turned on or off. To turn the light on a 1 should be inputted, to turn it off a 0 should be inputted. If the light is being turned on the corresponding bit on PORTB will be set to 1, whereas if its being turned on the bit will be set to 0.	These functions are called whenever the cardinal direction of the compass changes.
north east south west	These four functions are the main functions for displaying a new cardinal direction. They will first turn on the corresponding light for the direction, and then display the new cardinal direction to the LCD.	These are called by the changeDirection function, whenever the direction changes.

2.c.ii Compass Code

Communicating with the MMC3416PJ requires the use of I2C, to make this easier we used the I2C Master C-Library and implemented functions described in the MMC3416PJ's datasheet as well as in other existing libraries for the MMC3416PJ.

Function	Description	Use
getDegree	This function takes two pointers to lists of 3 floats that represent the 3-axis data from the sensors and the current offset calculated from compass_init. The process of getting a measurement from the MMC3416PJ involves telling the sensor to take a measurement, polling the status register until it is done taking measurements, then reading in registers 0-5 which store the low and high bytes for the 3 axes. The measurements are then subtracted by the offset to get which way the magnetic field is pointing. This data is then used to calculate the angle using arctangents. This function returns the angle in degrees.	Used for getting the current reading from the sensor and calculating the angle.
compass_init	This function is used to determine the offset of the sensor. First it tells the sensor to magnetize itself at 0 degrees. Then measurements are taken to determine that max forward value of the sensor. Then we repeat the process but set the	Used for reducing noise in the system and getting a reference measurement .

	sensor at 180 degrees to get maximum measurement in the opposite direction. These values are then averaged to get the center point of where the sensor would detect no magnetic field, this average is used as the offset for future readings. The sensor is then told to stop magnetizing itself.	
--	--	--

3 Experimental Methods

The implementation of our project, as stated many times above, was split into different components to avoid conflicts between components in initial coding and set up. This allowed us to completely set up each component fully and verify they worked before adding them all together at the end. By doing this, we eliminated overlapping errors from components to completely focus on implementing one component at a time. When implementing all of the display components we would begin with the LCD displaying power off, then if the PBS was pressed we would turn on a random LED and display a random direction and degrees to ensure that it all worked. Then, for the compass we tested using a BAUD connection to read the sensor data and to debug where the program got stuck while communicating with the chip. We also used multimeters and oscilloscopes to verify that our I2C communication and level shifter were properly working.

4 Results

The following pictures show the results of our compass. The first picture is an overview of our circuit, the second picture shows the LCD display, and the third picture shows the compass direction being displayed on the LCD.

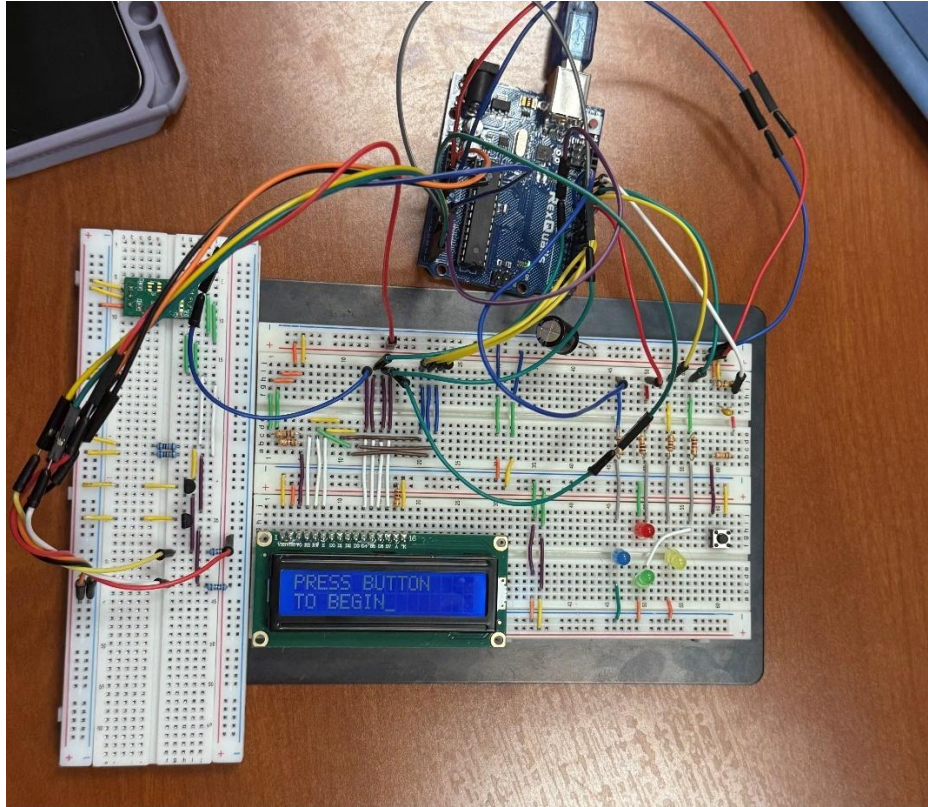


Figure 5: Overall Circuit



Figure 6: LCD Display

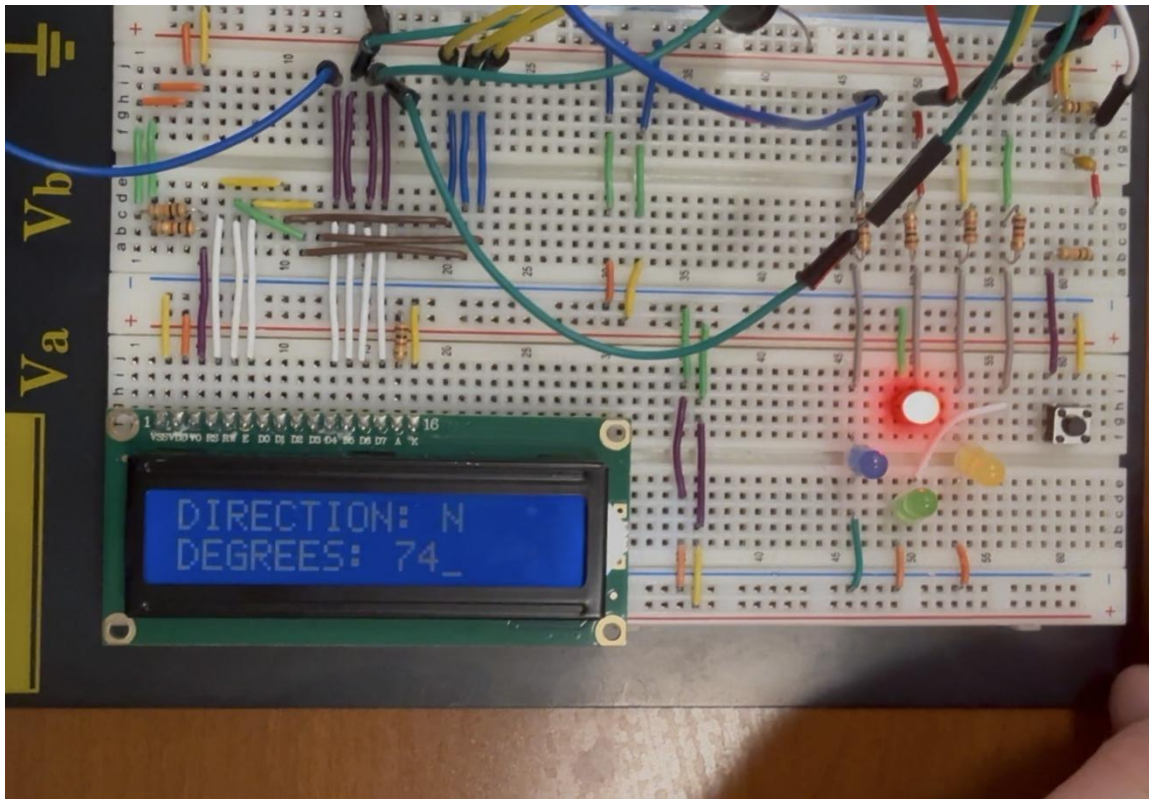


Figure 7: Showing Direction

5 Discussion

Overall, we are satisfied with the state we got our project in. We had issues getting the sensor to behave how we wanted, for it to give the heading in degrees, but we believe this was a problem with our sensor as using Arduino libraries to run the sensor were having the same issues. Additionally, we had some reliability issues with the code that we could have spent more time to fix, the major issue was that if the microcontroller or the MMC3416PJ missed an ack during communication the whole device would freeze and need to be restarted.

6 Conclusion

We learned a lot about I2C communication, and mixed voltage systems, as well as combining skills we learned from previous labs to create an interface for the device. If we were to continue with this project, we might add other I2C devices such as an accelerometer to get more positional data on the device to allow for more advanced calculations as well as making our device more robust to errors.

7 References

Peter Fleury's LCD Library: http://www.peterfleury.epizy.com/doxygen/avr-gcc-libraries/group_pfleury_lcd.html

Peter Fleury's I2C Library: http://www.peterfleury.epizy.com/doxygen/avr-gcc-libraries/group_pfleury_ic2master.html

MMC3416xPJ Datasheet:

<https://mm.digikey.com/Volume0/opasdata/d220001/medias/docus/1793/MMC3416xPJ.pdf>

MMC3416xPJ Reference Library:

<https://github.com/levkovigor/MMC34160PJ>

8 Source Code

```
/*
 * FinalProject.c
 *
 * Created: 4/24/2025 11:31:18 AM
 * Author : ellap
 */

#define F_CPU 16000000UL

#define ADDRESS 0b01100000
#define declination -1.19

#include <avr/io.h>
#include <math.h>
#include <string.h>
#include <util/delay.h>
#include <stdlib.h>
#include <stdio.h>
#include "lcd.h"
#include "i2cmaster.h"

void redLight(int status){ //controls red LED
    if(status == 0){
        PORTB &= ~ 0B00000010; //turn off
    }else if(status == 1){
        PORTB |= 0B00000010; //turn on
    }
}

void yellowLight(int status){ //controls yellow LED
    if(status == 0){
        PORTB &= ~ 0B00000100; //turn off
    }else if(status == 1){
        PORTB |= 0B00000100; //turn on
    }
}

void greenLight(int status){ //controls green LED
    if(status == 0){
        PORTB &= ~ 0B00001000; //turn off
    }else if(status == 1){
```

```

        PORTB |= 0B00001000; //turn on
    }
}

void blueLight(int status){ //controls blue LED
    if(status == 0){
        PORTB &= ~ 0B00000001; //turn off
    }else if(status == 1){
        PORTB |= 0B00000001; //turn on
    }
}

void north(void){ //turns on red light and displays N on LCD
    redLight(1);
    lcd_clrscr();
    lcd_home();
    lcd_puts("DIRECTION: N");
}

void east(void){ //turns on yellow light and displays E on LCD
    yellowLight(1);
    lcd_clrscr();
    lcd_home();
    lcd_puts("DIRECTION: E");
}

void south(void){ //turns on green light and displays S on LCD
    greenLight(1);
    lcd_clrscr();
    lcd_home();
    lcd_puts("DIRECTION: S");
}

void west(void){ //turns on blue light and displays W on LCD
    blueLight(1);
    lcd_clrscr();
    lcd_home();
    lcd_puts("DIRECTION: W");
}

void changeDirection(int currentDirection, int nextDirection){ //changes the direction
shown on LCD
    //(1 = N, 2= E, 3=S, 4=W)
    if(currentDirection == 1){
        redLight(0);
    }
    if(currentDirection == 2){
        yellowLight(0);
    }
    if(currentDirection == 3){
        greenLight(0);
    }
    if(currentDirection == 4){
        blueLight(0);
    }
    if(nextDirection == 1){
        north();
    }
    if(nextDirection == 2){

```



```

        east();
    }
    if(nextDirection == 3){
        south();
    }
    if(nextDirection == 4){
        west();
    }
}

void writeRegister(uint8_t reg, uint8_t val){
    i2c_start_wait(ADDRESS | 0x00);
    i2c_write(reg);
    i2c_write(val);
    i2c_stop();
}

void turnOffCompass(void){
    //turn off compass communication

    lcd_clrscr();
    lcd_home();
    lcd_puts("POWER: OFF");
    lcd_gotoxy(0,1);
    lcd_puts("PRESS BUTTON");

    redLight(0); // turn off all lights
    greenLight(0);
    yellowLight(0);
    blueLight(0);
}

int getDegree(float* offset, float* data){

    unsigned char temp;
    writeRegister(0x07, 0x01);

    int flag = 0;
    while (flag == 0){
        i2c_start_wait(ADDRESS | 0x00);
        i2c_write(0b00000110);
        i2c_stop();
        i2c_start_wait(ADDRESS | 0x01);
        temp = i2c_readAck();
        temp &= 1;
        if (temp != 0){
            flag = 1;
        }
    }

    uint8_t out[6] = {0,0,0,0,0,0};

    i2c_start_wait(ADDRESS | 0x00);
    i2c_write(0x00);
    i2c_start(ADDRESS | 0x01);
    for(int i = 0; i < 6; i++){
        out[i] = i2c_readAck();
    }
}

```

```

}
i2c_start(ADDRESS | 0x00);
i2c_stop();

unsigned int raw[3] = {0,0,0};

raw[0] = out[1] << 8 | out[0]; //x
raw[1] = out[3] << 8 | out[2]; //y
raw[2] = out[5] << 8 | out[4]; //z

float temp0 = 0;
float temp1 = 0;
float deg = 0;

for (int i=0; i<3; i++) {
    data[i] = 0.48828125 * (float)raw[i] - offset[i];
}
float axis1 = data[0];
float axis2 = data[1];
if (axis1 < 0) {
    if (axis2 > 0) {
        //Quadrant 1
        temp0 = axis2;
        temp1 = -axis1;
        deg = 90 - atan(temp0 / temp1) * (180 / 3.14159);
    } else {
        //Quadrant 2
        temp0 = -axis2;
        temp1 = -axis1;
        deg = 90 + atan(temp0 / temp1) * (180 / 3.14159);
    }
} else {
    if (axis2 < 0) {
        //Quadrant 3
        temp0 = -axis2;
        temp1 = axis1;
        deg = 270 - atan(temp0 / temp1) * (180 / 3.14159);
    } else {
        //Quadrant 4
        temp0 = axis2;
        temp1 = axis1;
        deg = 270 + atan(temp0 / temp1) * (180 / 3.14159);
    }
}

deg += declination;
if (declination > 0) {
    if (deg > 360) {
        deg -= 360;
    }
} else {
    if (deg < 0) {
        deg += 360;
    }
}
return (int)deg;
}

```

```

void compass_init(float* offset){
    float* data1 = (float*)malloc(3 * sizeof(float));
    float* data2 = (float*)malloc(3 * sizeof(float));

    writeRegister(0x07, 0x80);
    _delay_ms(60);
    writeRegister(0x07, 0x20);
    _delay_ms(10);
    getDegree(offset, data1);
    writeRegister(0x07, 0x80);
    _delay_ms(60);
    writeRegister(0x07, 0x40);
    _delay_ms(10);
    getDegree(offset, data2);

    for (int i=0; i<3; i++) {
        offset[i] = (data1[i]+data2[i])*0.5;
    }

    writeRegister(0x07, 0x40);
    _delay_ms(10);
    writeRegister(0x07, 0x00);
    _delay_ms(10);

    free(data1);
    free(data2);
}

int getDirection(int degree){ //return the direction as an int
    int compassDirection = 0; //holds int to return to program to display outputs

    if(degree < 90){
        compassDirection = 1;
    }else if((degree >= 90) & (degree < 180)){
        compassDirection = 2;
    }else if((degree >= 180) & (degree < 270)){
        compassDirection = 3;
    }else if(degree >= 270){
        compassDirection = 4;
    }
    return compassDirection;
}

void changeDegree(int newDegrees){ //displays new degree to LCD
    char degreeString[20];
    sprintf(degreeString, "%d", newDegrees);
    lcd_gotoxy(0,1);
    lcd_puts("DEGREES: ");
    lcd_gotoxy(9,1);
    lcd_puts(degreeString);
}

void turnOnLCD(void){ //initializes and turns on LCD
    lcd_init(LCD_DISP_ON_CURSOR); //initialize LCD and turn on
    lcd_clrscr(); //clear screen
    lcd_home(); //put cursor in home position
    lcd_puts("PRESS BUTTON"); //power for compass initially off
    lcd_gotoxy(0,1);
}

```



```

        lcd_puts("TO BEGIN"); //will power on with button press
    }

    int main(void)
    {
        DDRD |= 0B11100000; // PD5-PD7 are all outputs (used for LCD; E, RS, and R/W)
        DDRB |= 0B00001111; //PB0-PB4 are all outputs, PB6 is an input (used for PBS and
LEDs)
        DDRC |= 0B00111111; //PC0-PC4 are all outputs (used for LCD; D4-D7)

        float* offset = (float*)malloc(3 * sizeof(float));;
        float* data = (float*)malloc(3 * sizeof(float));;

        offset[0] = 0;
        offset[1] = 0;
        offset[2] = 0;

        i2c_init();
        turnOnLCD();
        int currentDirection = 0;
        int nextDirection = 0;
        int currentDegree = 0;
        int nextDegree = 0;
        int power = 0;
        offset[0] = 0;
        offset[1] = 0;
        offset[2] = 0;
        compass_init(offset);
        while(1){ //loop

            unsigned char temp = PINB; //get PINB
            temp = temp & 0B00100000; //mask to only get button input
            if((temp != 0B00100000) & (power == 0)){ //check if button pressed
                power = 1;
                while((PINB & 0B00100000) != 0B00100000); //wait for button debounce
            }
            else if((temp != 0B00100000) & (power == 1)){ //if button pressed and power
is on then turn off
                turnOffCompass();
                power = 0;
                currentDegree = 0; //reset directions and degrees
                nextDegree = 0;
                nextDirection = 0;
                currentDirection = 0;
                while((PINB & 0B00100000) != 0B00100000); //button debounce
            }
            if(power == 1){ //if power is on and button not pressed constantly get
direction and degrees
                nextDegree = getDegree(offset, data);
                nextDirection = getDirection(nextDegree);
                if(currentDegree != nextDegree){ //if degree changed update LCD
                    changeDirection(currentDirection, nextDirection);
                    changeDegree(nextDegree);
                    currentDirection = nextDirection;
                    currentDegree = nextDegree;
                }
                _delay_ms(10);
            }
        }
    }

```

}
}